THOMAS MORE

Cloud & IoT Monitoring and Data Analysis with Elasticsearch and Grafana

Realization document

Birate Kabanza Arthur

Student Bachelor Applied Computer Science

2024 – 2025

Table of Contents

1. Introduction

This realization document marks the conclusion of my internship at Van Genechten Packaging (VGP). The internship was part of my bachelor's degree in applied computer science at Thomas More University during the 2024-2025 academic year. This experience allowed me to use my educational knowledge in a real-world environment.

This internship assignment is a continuation of my initial project plan that aimed at creating a Monitoring and Data Analysis Framework of Cloud and IoT. The primary purpose was to develop the real-time monitoring systems of VGP to enhance their operations and improve decision-making using accurate information. The framework aimed to enable quicker responses to infrastructure problems through improved monitoring and analysis. Under this assignment, I have used the standard given tools, Grafana, Elasticsearch, and Kibana, to design a scalable monitoring system that scales with the company's growth.

This document is divided into sections:

Internship Context

The section provides an overview of Van Genechten Packaging, which operates as a leading packaging manufacturer. The section emphasizes the critical role of monitoring and analytics for its private cloud setup. System visibility and performance tracking serve as essential elements for maintaining stable IT and production operations.

The tools and technologies Applied

This project included Elasticsearch, Kafka, Logstash, Grafana, Kibana, Elastic Agent, Filebeat, and Metricbeat. It explains what each tool does, why it matters to the structure, and how they were combined into a monitoring and analytics solution.

Technical Research and Comparison

The following section contains my research findings about alternative monitoring technologies. The comparison between Elasticsearch, Prometheus, Kafka, RabbitMQ, Grafana, Power BI, and OpenSearch helped me identify their respective advantages, disadvantages, and trade-offs. The research supports the chosen technology stack and demonstrates my ability to learn beyond the internship requirements.

Analysis

The evaluation of this section assesses the monitoring stack's advantages and disadvantages, together with its potential risks. The selection of Elasticsearch, Kafka, Logstash, Kibana, and Grafana for VGP's monitoring framework is justified through their ability to handle real-time data, provide unified observability and anomaly detection, and flexible visualization. The section presents the primary limitations and architectural risks encountered during implementation, as well as alternative solution considerations.

Implementation

The following section discusses the development process of the monitoring framework along with its deployment details. The system architecture is examined alongside data collection and ingestion methods, data stream processing and transformation, and Elasticsearch storage and indexing strategies. The explanation emphasizes how Kafka, Logstash, Elasticsearch, and Beats/Elastic Agents were integrated to create a reliable pipeline that supports real-time monitoring and visualization.

Realization – Visualization Layer

The following section demonstrates the dashboards I built using Grafana and Kibana to support both developers and operators at the company. The section describes the function of each dashboard together with the changes implemented to fulfill user requirements and their benefits for delivering understandable, responsive, and actionable insights. The dashboards provide technical staff and non-technical users with effective system monitoring capabilities, which enable them to make swift, informed decisions.

Results and benefits

The section presents a summary of the project's accomplishments, which include real-time monitoring and fast unusual activity detection, as well as business stability enhancement, faster issue resolution, and improved collaboration between technical teams and production staff. And enhanced collaboration between technical teams and production staff.

Conclusion

The final section evaluates the internship project's success through both technical achievements and individual development. The project evaluation assesses its contribution to VGP's operational objectives while summarizing key learning points from the internship experience, including problem-solving abilities, critical thinking, and working in demanding technical settings.

2. Internship Context

The European market leader Van Genechten Packaging (VGP) has established itself through its innovative, sustainable packaging solutions and new business ideas. The company needed to update its IT infrastructure to support its digital transformation as part of its organizational strategy. The company selected OpenShift and Open Kubernetes Distribution to establish a private cloud system. The platform enables organizations to operate and control extensive container applications. The platform enhances operational efficiency through its scalable and flexible architecture.

Internship Goal

Cloud and IoT monitoring with advanced data analysis is the primary organizational focus. VGP, using its private cloud system, implemented Elasticsearch and Grafana to gather operational data from IoT production tools, virtual machines, and cloud applications. Elasticsearch's data management capabilities offer users quick access to system logs, performance metrics, and production data. Grafana creates interactive dashboards that allow users to monitor infrastructure status, detect issues, and analyze production performance.

The combination provides predictive maintenance capabilities while enhancing production operations and delivering data-driven insights that enable informed decisions. Through this framework, VGP achieves better quality control while enhancing sustainability initiatives and market adaptation speed.

I joined the IT team to resolve the main problem of maintaining operational stability for VGP's private cloud and IoT systems during my internship period. The primary goal involved designing a complete monitoring and reporting framework that covered hardware infrastructure as well as network systems, including virtual machines and microservices.

The system uses Elasticsearch to index and search data and Grafana to develop interactive visualizations. The established framework delivers immediate performance insights about cloud infrastructure and IoT production system operations. Early problem detection, along with reduced downtime risk and enhanced operational decision-making capabilities, results from this setup. Teams achieve faster problem resolution, improved resource utilization, and sustained production operations through dashboard-based log data, metrics, and alert presentation.

The internship assignment involved implementing application metrics and IoT device data monitoring, which expanded monitoring capabilities beyond server and network systems. The monitoring process required continuous performance checks of microservices, together with log assessment and production equipment data retrieval.

Logstash served as the essential component of the ELK stack, so I dedicated substantial time to gaining practical experience with this tool during my job. Through Logstash, I constructed data pipelines to handle real-time processing of extensive application and IoT data for both intake, modification, and distribution. This process ensured all vital data remained accessible for analysis and display.

3. Tools and Technologies Applied

During the internship, a variety of tools and technologies were used to develop an effective cloud and IoT monitoring and data analysis framework. These technologies were selected to ensure scalability, real-time data processing, and visualization, enabling quick insights.

Elasticsearch

Elasticsearch functions as a distributed search and analytics engine that specializes in storing and indexing massive amounts of data, including both structured and unstructured information. The system divides data into indexes, which contain documents that are further divided into searchable fields that can be aggregated. The inverted index allows fast searching of extensive datasets.

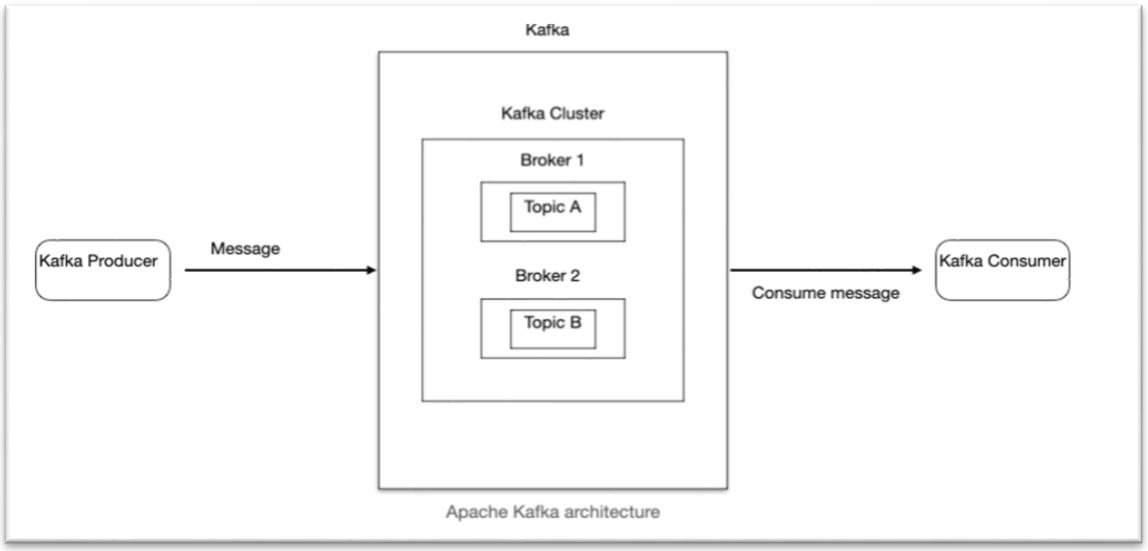
The central data storage system used was Elasticsearch, which handles structured data streams that Logstash and Elastic agents sent to it. The aggregation features of the system enabled the development of real-time summaries and the identification of trends within logs and performance metrics. The machine learning module of Elasticsearch was employed to detect anomalies in time-series data. The system eliminated the requirement for fixed thresholds because it enabled the detection of abnormal system behavior at an early stage and adaptive monitoring capabilities.

Elasticsearch was selected because it scales effectively. offers strong search and aggregation capabilities, and includes machine learning. It can handle both logs and metrics in one place, which makes data management simpler and provides complete insights into how the system is running.

Kafka

Kafka operates as a distributed streaming platform that processes real-time data streams. The system serves as a highly scalable message broker that enables systems to exchange large amounts of data effectively. The system organizes incoming data into topics, which are further divided into partitions to allow parallel processing and maximize output.

Kafka producers sent logs and metrics to Kafka topics at regular intervals. The broker design of Kafka maintains separate producer and consumer systems, which store data in partitions until processing occurs. The asynchronous data delivery system of Logstash and other consumers allows for reliable data processing from collection to analysis without any data loss.



Kafka was chosen because it can manage large amounts of data coming from many IoT devices, virtual machines, and apps. Its design allows it to grow, so the platform can take in and send out real-time data without slowing down or losing messages, even when the amount of data increases.

Elastic Agents

The Elastic Agents gathered system data, which included CPU usage, memory usage, network flow, and logs from different sources. The agents transmitted data to Logstash or Elasticsearch for storage before processing occurred. The design established a uniform system for collecting data, which operated at a centralized level and maintained scalability throughout the entire tech infrastructure.

Kibana

Kibana is a data exploration and visualization tool made to work with Elasticsearch. It has good search and log analysis features, plus dashboards that can be changed to watch and understand Elasticsearch data.

In this project, Kibana was utilized to analyze logs and identify the causes of problems. The query interface enabled the IT and DevOps teams to sort and search large log datasets, easily pinpoint errors, and correlate events from different sources. This made troubleshooting easier and faster.

Grafana

The tool enables users to develop interactive dashboards for monitoring and tracking operational data. The system performance can be displayed in real-time through Grafana by integrating data from different sources, including Elasticsearch, with customized visualizations.

The monitoring system was able to establish a connection with Elasticsearch indices through Grafana. The system enabled the creation of dashboards that used specific data streams to fulfill the requirements of developers and operations staff. These dashboards allow the analysis of performance trends, anomaly detection, and resource usage monitoring for the entire cloud infrastructure and IoT devices.

4. Technical Research and Comparison

The realization phase required both the implementation of chosen tools and their evaluation against other available options. This section presents a comparative study where the tools used at Van Genechten Packaging (VGP) are evaluated alongside alternative technologies. The comparison is based on research done during the project and looks at the strengths, weaknesses, and how well each solution fits within the VGP monitoring framework.

4.1 Elasticsearch vs Prometheus

Feature	Elasticsearch	Prometheus
Primary Use Case	Search, log analytics, anomaly detection	Metrics-based monitoring and alerting
Data Model	Document-oriented (JSON), supports both structured and unstructured data	Time-series metrics with key-value labels
Query Language	Elasticsearch Query DSL	PromQL
Real-Time Processing	Excellent for logs and events; supports streaming data	Excellent for metrics; optimized for fast aggregation
Anomaly Detection	Built-in machine learning for unsupervised anomaly detection	No native anomaly detection; relies on rule-based alerts
Scalability	Highly scalable for large log datasets	Horizontal scaling is possible but more complex

Elasticsearch and Prometheus are both common choices for monitoring, but they suit different data and situations. Elasticsearch is very good at storing and searching through large amounts of log data. It gives a strong search and analytics ability for text (Services, What is the ELK Stack?, n.d.). Prometheus is made for collecting time-series data and sending alerts, which makes it a good fit for keeping track of real-time performance and working with number-based data. (Signoz, Prometheus vs Elasticsearch stack – Key concepts, features, and differences, 2023)

In conclusion, the research points to Elasticsearch as the better leading data storage solution for VGP's operations. This is due to its ability to handle log and metric collection, provide advanced search, and support machine learning for anomaly detection. Elasticsearch suits the company's monitoring needs because it can index and query large amounts of different log data, as well as support visualizations and anomaly detection. Prometheus could still be useful as a supplement, especially for frequent, short-term numeric metrics where

simple scraping and alerts are helpful. Even so, it cannot replace Elasticsearch for in-depth log analysis or complex searches.

4.2 Kafka vs RabbitMQ

Feature	Kafka	RabbitMQ
Messaging Model	Distributed commit log, publish-subscribe	Message queue (AMQP)
Data Persistence	Retains messages for a configurable period, even if consumed	Messages typically deleted once acknowledged
Throughput	Very high; ideal for big data pipelines	Lower throughput, but good for transactional workloads
Ordering	Maintains order within a partition	Maintains order per queue
Scalability	Designed for horizontal scaling	Supports clustering but less suited for massive scale
Use Case Fit	Event streaming, IoT data ingestion, log collection	Task distribution, point-to-point messaging

The two most used message brokers, Kafka and RabbitMQ, serve different messaging patterns and operational requirements. Kafka operates as a streaming platform that specializes in processing large amounts of real-time data and event-driven systems (Simplilearn, 2023). Kafka works best in scenarios where the order of messages is essential and must be preserved, and numerous messages require proper storage and replay functionality. The system's durability features and scalability capabilities make Kafka suitable for big data environments, as well as data analysis and log collection applications.

The project required Kafka as its best solution because it needed to process large volumes of streaming logs and IoT data. The project required Kafka because of its high throughput capabilities, partitioning model, and real-time event stream handling. Choice. The system processed and stored data streams effectively because of its durability and scalability features. RabbitMQ provides better performance when applications require transactional job queues, message delivery guarantees, and complex routing patterns (Simplilearn, 2023). The system requires low latency and flexible message handling capabilities above throughput performance in these specific use cases.

4.3 Grafana vs Power BI for Real-Time Monitoring

While Power BI is a popular BI tool for analyzing historical business data, its capabilities differ from Grafana in infrastructure monitoring. Grafana excels at real-time metrics, alerts, and operational monitoring, whereas Power BI focuses on analytical reporting and strategic insights (MetricFire, 2023).

Feature	Grafana	Power BI
Real-Time Updates	Near real-time with supported data sources	Primarily batch refresh; near real-time requires special setup
Integration with Monitoring Tools	Native support for Elasticsearch, Prometheus, InfluxDB, etc.	Stronger for relational databases, Excel, cloud analytics
Dashboard Interactivity	Highly customizable, plugin-based	Good interactivity, but less flexibility for metrics dashboards
Ordering	Maintains order within a partition	Maintains order per queue
Deployment	Open-source, on-prem or cloud	Cloud-centric, on-prem via Power BI Report Server
Cost	Free (open source) or low cost for enterprise features	Licensing costs at scale

In conclusion, Grafana's native integration with Elasticsearch and other monitoring sources, along with its real-time update capabilities, makes it highly suitable for VGP's infrastructure monitoring needs. While Power BI remains valuable for business-level reporting and analysis, it is less ideal for continuous, real-time monitoring of systems and applications.

Summary of Findings:

The technology stack, which includes Elasticsearch, Kafka, Grafana, and Kibana, proved helpful through practical application and comparison with other options. Each part has a key function: Elasticsearch allows strong search and analytics for logs and metrics; Kafka ensures high-throughput streaming and dependable data pipelines; and Grafana and Kibana provide easy-to-understand, real-time visualization and monitoring capabilities. Even though other tools might be better in certain situations, this stack offers the optimal balance of scalability, flexibility, and real-time speed to support VGP's private cloud infrastructure and IoT monitoring framework, guaranteeing data processing, constant observability, and practical insights for the organization.

Elasticsearch vs OpenSearch

OpenSearch, which started as a version of Elasticsearch 7.10, shares many of its basic features as an open-source option (Coralogix, 2024). For this project, OpenSearch didn't have built-in machine learning tools. So, to get similar anomaly detection, we would've needed to use outside extensions or create something ourselves.

Because the project required dependable, built-in anomaly detection that integrated seamlessly with the Elastic system, including Kibana, Logstash, and Beats, Elasticsearch was chosen. It seemed like a better, more complete technical match for what we needed at the time**.**

The IT team did a detailed review of monitoring and analytics tools to find the best ones for their cloud and IoT monitoring setup. They chose Elasticsearch, Kibana, and Grafana instead of options like Power BI for these main reasons:

- They give better real-time monitoring and reporting, which is key for keeping their cloud setup and microservices running without interruption.
- They easily work with their private cloud setup (OpenShift/Kubernetes), helping gather data from containers, VMs, and IoT devices.
- They can handle large amounts of time-based and log data across different layers of their system.
- Being open-source and able to be set up on-site gives them total control over data security and their setup.
- They allow for detailed customization and visualizations that offer valuable data through custom dashboards.

5. Analysis

5.1 Critical Evaluation of Chosen Stack

A monitoring system relies on various tools to handle large data volumes, analyze efficiently, and present useful visual information. Elasticsearch, Kafka, Logstash, Kibana, and Grafana were selected. They work together effectively to collect, process, and display data, offering quick insights and real-time analysis capabilities for VGP's private cloud and IoT monitoring setup.

Justification for Selection:

Real-Time Data Management:

Elasticsearch, along with Kafka, enables quick ingestion and querying of logs and metrics. Kafka's high capacity and partition-based architecture ensure reliable handling of IoT and infrastructure telemetry without data loss.

Unified Observability:

By integrating logs, metrics, and anomaly detection into a single Elastic Stack setup, VGP eliminated the challenge of managing multiple systems for different data types.

Machine Learning Integration:

Elasticsearch's built-in anomaly detection enabled pattern recognition directly on operational data without supervision, reducing the reliance on static thresholds.

Visualization Choices:

Kibana provides deep search and log analysis features, while Grafana offers highly customizable metric dashboards. This combination enables tailored solutions for both technical and non-technical users.

Open Source and Local Hosting:

The open-source nature of the selected tools, along with the ability to run them locally, provided complete control over sensitive operational data, which was a key feature for VGP's private cloud setup.

Trade-offs Considered:

- Choosing Grafana alone makes managing visualizations simpler, but you would lose Kibana's powerful search capabilities.
- Switching from Kafka to RabbitMQ makes managing transaction messages easier. However, RabbitMQ is less effective at handling large, continuous data streams.

- Using Prometheus with Elasticsearch can improve managing high-frequency metrics, but it also increases complexity and maintenance challenges.

5.2 Limitations and Risks

While the chosen architecture proved effective in meeting the core monitoring requirements, several limitations and potential risks were identified during implementation. These included considerations around system scalability under peak loads, maintenance complexity, integration challenges with specific data sources, and the need for ongoing optimization to ensure consistent performance and reliability.

Tools-specific Limitations

Elasticsearch

- Storage Costs**:** Rapid index growth requires strict lifecycle management to control infrastructure expenses.
- Operational Complexity**:** Cluster scaling and shard management demand specialized skills.
- Machine Learning Requirements**:** Anomaly detection models need adequate historical data to be reliable.

Kafka

- Management Overhead**:** Requires ongoing monitoring of broker health, topic partitioning, and replication factors.
- Ordering Constraints**:** Guarantees ordering only within partitions, requiring careful design for correlation-sensitive data.

Grafana and Kibana

- Overlapping Functions**:** Maintaining both platforms increases administrative workload.
- Learning Curve**:** Non-technical users may require dedicated training to navigate dashboards effectively.

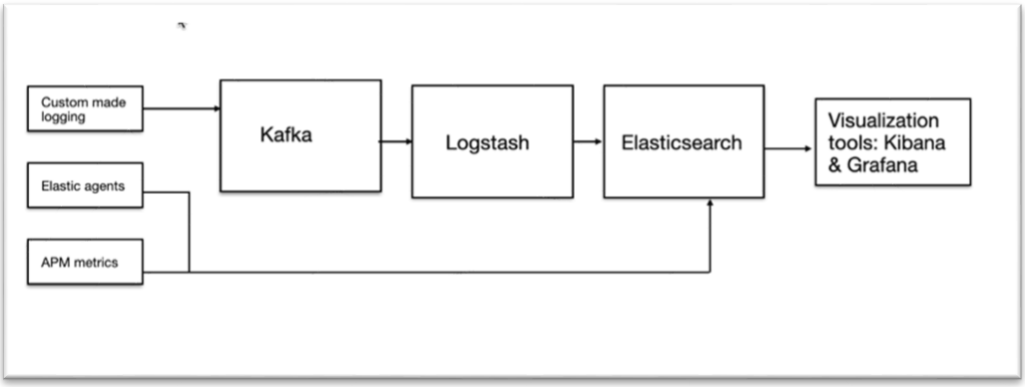
Architectural Risks

- Mapping Conflicts**:** Without strict index templates, schema changes could cause query failures.
- Network Latency Dependence**:** Real-time performance depends on stable, low-latency connections between ingestion, storage, and visualization layers.

6. Implementation

6.1 System Architecture Overview

Data was gathered from different parts of the setup, like servers, virtual machines, IoT gadgets, and production systems. Data shippers were used to collect and send specific monitoring data to the primary data flow.



6.2 Data Collection and Ingestion

Elastic Agent

It was used to collect system metrics, application logs, and other observability data from servers and services. Once installed, the Elastic Agent gathered the configured data stream, such as CPU and memory usage, disk I/O, network activity, and application log, and securely sent them to Elasticsearch. The agent handled both collection and transmission, ensuring that data from multiple sources was consistently formatted and available for search, analysis, and visualization in Kibana.

APM Metrics

They were collected by APM agents running within the applications to monitor performance and resource usage. These metrics included request latency, throughput, error rates, database query times, memory usage, and CPU consumption. The APM agents continuously gathered this data and sent it to the APM Server, which processed and forwarded it to Elasticsearch. This provided real-time visibility into application health and performance, enabling teams to detect issues quickly, identify bottlenecks, and optimize overall system efficiency.

Filebeat

It was used to collect log files generated by applications, services, and systems. Filebeat continuously monitored these files for new entries, performed basic parsing and filtering, enriched the logs with additional metadata, and sent the data directly to Elasticsearch. This ensured that log information was always available for search, analysis, and visualization.

Metricbeat

It was used to gather system-level and service-level metrics, such as CPU usage, memory consumption, disk utilization, and service-specific statistics like Ceph storage metrics. The collected data was sent directly to Elasticsearch, providing real-time visibility into infrastructure performance and resource usage.

Kafka Topics

They were used to organize incoming data streams based on type, such as system metrics, logs, or IoT data. Data producers, such as Elastic Agents and Beats, sent information to the appropriate Kafka topic, allowing consumers like Logstash to subscribe only to the relevant data. This approach ensured efficient routing and processing of large-scale data.

Kafka Brokers

They acted as the servers responsible for managing Kafka topics and storing messages reliably. Kafka brokers ensured that messages were replicated for fault tolerance and could be retrieved by consumers when needed. Data producers, including Elastic Agents and Beats, sent messages to the brokers, while consumers like Logstash retrieved the data for further processing and indexing in Elasticsearch.

6.3 Data Processing and Transformation

After data is gathered and put into Kafka, Logstash processes and prepares the data before indexing it into Elasticsearch, Logstash is a flexible data processing system with a modular design, having three main parts: input, filter, and output.

Input Stage

Logstash gets data from different sources like Kafka topics, Beats, and Elasticsearch indexes (when Elasticsearch is a data source). This lets Logstash gather streaming data from many agents and systems right away.

Filter Stage

Here, Logstash changes raw data to make it more usable. Parsing filters turn unstructured logs into structured fields. Enrichment filters add metadata to events, like system names or environment tags. Drop filters remove unnecessary information, which reduces storage use and improves search results.

Output Stage

Logstash then sends the changed data to Elasticsearch. After being indexed, the data can be quickly searched, grouped, and viewed using tools such as Kibana and Grafana.

6.4 Data Storage and Indexing

After Logstash processes the data, all structured information is sent to Elasticsearch for storage and indexing. Elasticsearch then puts this data into distributed indices, which allows for quick full-text searches, filtering, and real-time data summaries, even when working with large amounts of log and metric data. Logs, metrics, and IoT data are kept in different indices based on their type and when they were created. This setup makes it easy to find both current and past data.

In Elasticsearch, there are two options for indexing incoming data: using index templates to define the indexing process or using the dynamic mapping feature for automatic handling.

Explicit Mapping (Index Templates)

Explicit mappings involve predefining how incoming data should be stored.

- They specify how fields should be mapped (defining data types for each field), index settings (like the number of shards and replicas), and index naming patterns.
- Templates help keep things consistent, ensuring fields always have the right type (for example, a timestamp is always stored as a date, and numeric fields are stored as numbers).
- They prevent mapping problems that can occur when new data arrives with slight differences.
- They give total control over how storage is and how resources are used.

For this project, explicit mappings were set up for different data sources to ensure consistent indexing across all indices. Also, when planning the pipeline to the node graph index, explicit field mappings were made to ensure that data types were correct and to avoid mapping problems. This approach improved query speed and maintained stable visual performance.

Dynamic mapping

Let Elasticsearch index data even when you don't have a template set up. It looks at the first document and guesses the type of data in each field. This is useful for simple projects, but it can be risky when you're dealing with a lot of data in a business setting, especially if that data changes a lot.

If later documents have fields with types that don't match what Elasticsearch first guessed (like a text string where it expected a number), you can run into problems. These problems can cause:

- New documents are not being indexed correctly.
- Search queries can return wrong information or break completely.
- Errors when trying to group data in dashboards and reports.

While dynamic mapping is easy to begin with since it requires no setup, it's usually not appropriate for large, complex systems where the data changes rapidly.

To ensure data consistency, prevent mapping errors, and facilitate smooth analysis and chart creation, this project used index templates. These templates specify the data type for each field, aiding data ingestion and improving query performance.

7. Realization: Visualization Layer

7.1 Kibana Dashboards

Node Graph Optimization Pipeline

Purpose:

To deliver fast, interactive visualizations that map the relationships between virtual machines, network protocols, IP addresses, and operating systems. This pipeline enables rapid analysis of infrastructure communication patterns, helping identify connectivity issues, potential bottlenecks, and anomalous behavior within the IT environment. This pipeline takes logs from the main index (endpoint.events.network-default).

Challenges Faced:

- Slow Load Times: Complex node graphs often took 20–25 seconds to render, impacting user experience and analysis speed.
- Mapping Conflicts: Conflicting mappings in the data occasionally caused query errors, leading to incomplete or incorrect visualizations.
- Reverse DNS Lookup Failures: Many internal IP addresses could not be resolved due to security restrictions, limiting the ability to label and interpret nodes fully.

Solutions Implemented:

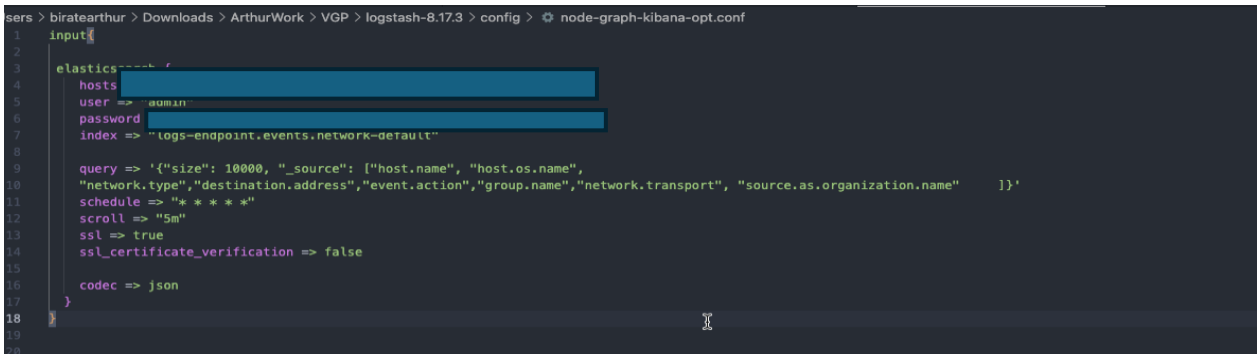
- Designed a dedicated Logstash pipeline to extract only essential fields for visualization.
- Applied explicit index templates to enforce consistent field typing.

Results:

- Improved Performance: Load times for complex node graphs were reduced to 5–8 seconds, achieving approximately a 70% improvement.
- Enhanced Stability: Mapping conflicts were resolved, resulting in 100% query reliability and consistent visualizations.
- Actionable Insights: IT teams gained a dependable tool to analyze IPv4/IPv6 distribution and identify suspicious external connections, improving overall infrastructure monitoring and security awareness.

In the input stage

Data was retrieved from Elasticsearch using a targeted query designed to capture only the most relevant fields, such as hostnames, IP addresses, network transport protocols, operating system details, and other attributes necessary for the node graph. This pre-filtering step ensured that unnecessary data was never pulled into the processing pipeline, minimizing overhead and optimizing throughput from the very start.



In the filter stage

I attempted to find domain names for the destination IP addresses through reverse DNS lookups. But none of the IP addresses had a valid domain name. This is likely because of security or missing reverse DNS records, or our view of the internal network's DNS is limited. Because of this, we didn't get any new context for the data. The node graph just used the original IP address fields to display things.

In the output stage

The optimized dataset was stored in a dedicated Elasticsearch index created explicitly for the node graph visualization. Before ingestion, a well-defined index template was applied to enforce precise field mappings, ensuring consistent data types and avoiding mapping conflicts. This approach not only maintained data integrity but also significantly improved Kibana's node graph loading performance, even as the volume of incoming logs continued to increase.

```
output{

    elasticsearch{
        hosts => [
        user =>"admin
        password
        index => "node-graph-vms-network-connection"
        ssl => true
        ssl_certificate_verification => false
    }


    stdout{
      codec => rubydebug
    }
```

Follow link (cmd + click)

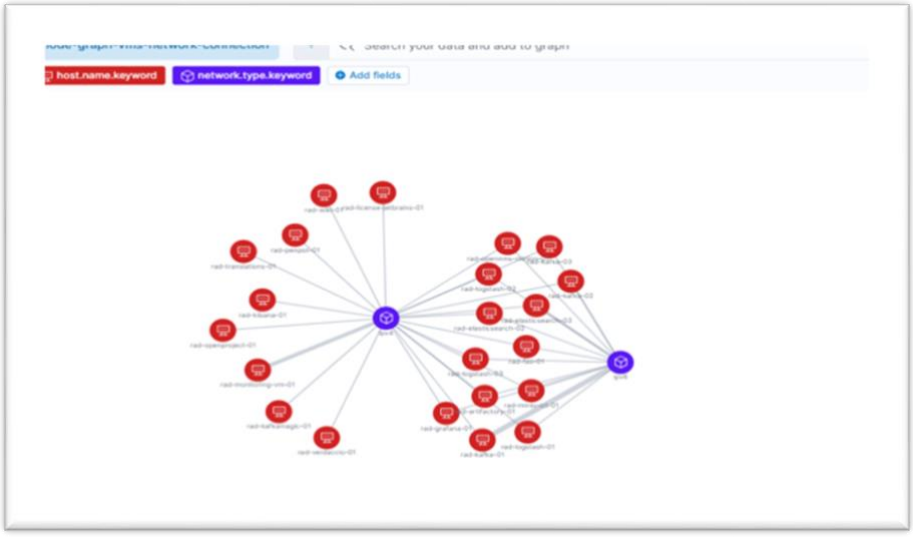
Cloud Infrastructure Node Graph

Purpose:

To provide IT teams with a real-time, interactive visualization of the private cloud infrastructure, enabling them to quickly understand communication patterns, dependencies, and potential problem areas. The graph helps improve system analysis, troubleshooting, and overall infrastructure management.

The Cloud Infrastructure Node Graph displays how virtual machines are interconnected, showing hostnames, IP addresses, operating systems, and network protocol details. It allows users to explore communication paths and dependencies across the environment, offering a clear and comprehensive view of the system architecture. This visualization supports rapid identification of bottlenecks, misconfigurations, or anomalous connections, ultimately enhancing performance monitoring and proactive maintenance.

1. Virtual Machine to Network Protocol

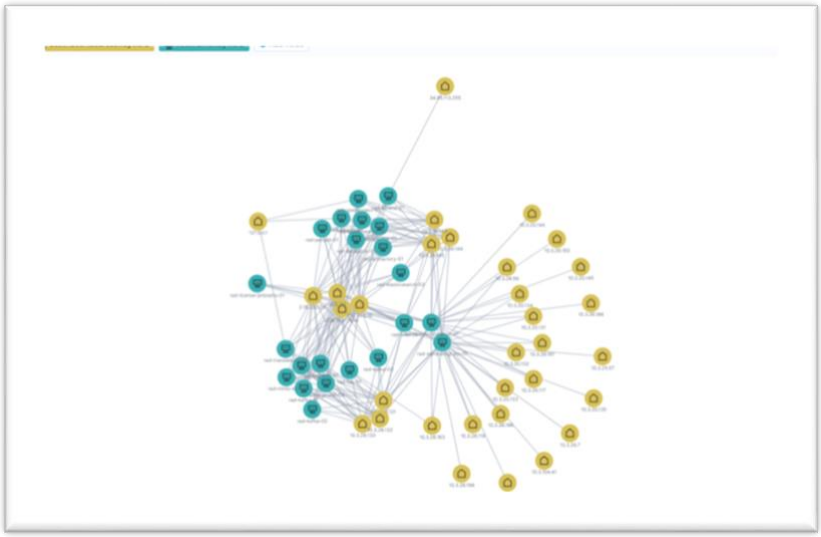


This node graph shows how each virtual machine, and the network protocols it uses, IPv4, IPv6, or both, are related.

- Virtual machine nodes are labeled with their hostnames, so you can easily see each system.
- Network protocol nodes are marked as either IPv4 or IPv6, showing what protocol is being used.
- The links between the virtual machine nodes and the protocol nodes tell you which network stack each system uses to communicate.

By showing this node graph, the IT team quickly figures out how protocols are spread across the infrastructure. It's easy to see which systems still only use IPv4, which have switched to IPv6, and which use both. This is useful for fixing connection problems, planning protocol updates, and making sure network settings are consistent.

2. Virtual Machine to IP address



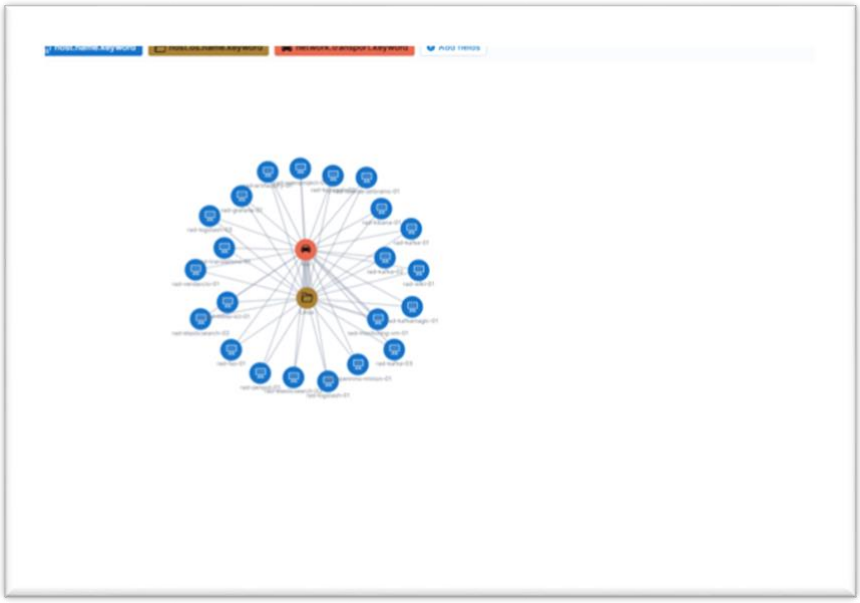
This node graph shows how each virtual machine (VM) in the network communicates with different IP addresses. Each VM is connected to the IP addresses it sends information to, creating a clear graph of all outgoing connections.

By visualizing these interactions, IT teams gain a deeper understanding of how the network operates and identify communication patterns. This enables them to see which external systems or services VMs are utilizing, highlighting both regular traffic and any suspicious or unauthorized connections. This is particularly helpful in identifying anomalies, such as VMs communicating with unknown IP addresses, which may indicate errors, security issues, or malware.

Also, this view helps with planning and improving the network. By looking at how much traffic goes between VMs and their IP addresses, teams make good choices about firewall rules, bandwidth, and resources. It also helps with checking and following rules, as the graph can show that all outgoing traffic fits with company rules and legal needs.

In short, the VM to IP Address node graph is a key tool for monitoring the network, checking security, and ensuring smooth operation. It gives IT teams a simple view of how VMs interact with other systems on the network, enabling them to act.

3. Virtual Machine to Operating System to Network Transport



This node graph shows how all the virtual machines, the operating system, and the network protocol are related across the whole system.

Each virtual machine is linked to a Linux node, which means that the cloud has a consistent OS environment. The graph then shows Linux connecting to the TCP node, which means that all network communication is done using the TCP protocol.

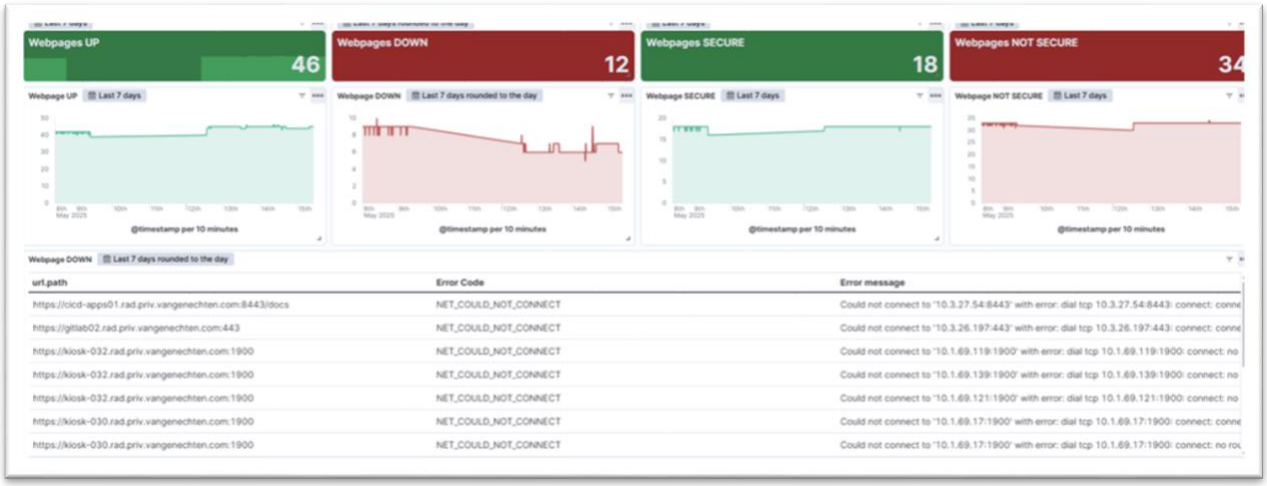
This node graph gives the IT team a good way to see how all the virtual machines connect to the same operating system and network protocol. Seeing these consistent links allows the team to check that all systems are set up the same way. This helps them quickly spot any unusual changes, reduces setup errors, and makes management easier.

To improve the performance of these graphs, a Node Graph Optimization Pipeline was created. The pipeline extracts only the essential information from the entire index and stores it in a separate index using specific mappings. This significantly increased the node graphs' loading speed, even with larger data loads.

Web Pages Status Dashboard

Purpose:

To continuously monitor the operational status and protocol security (HTTP/HTTPS) of all registered web services, providing real-time visibility into uptime and potential vulnerabilities. This enables early detection of service outages, identification of unsecured endpoints, and faster response to issues that could impact availability or security.



Besides monitoring protocols, the page tracks response times, HTTP status codes, and uptime. This allows teams to identify problems or downtime before users experience issues. By displaying past trends and alerting to unusual activity, it assists in planning maintenance and determining needed storage. The page also verifies if web services adhere to security standards, such as using secure HTTPS.

Overall, the Web Pages Status is a valuable tool for IT. It combines security monitoring with insights into the reliability of services. It helps teams keep web services running smoothly, fix issues quickly, and ensure your setup remains secure.

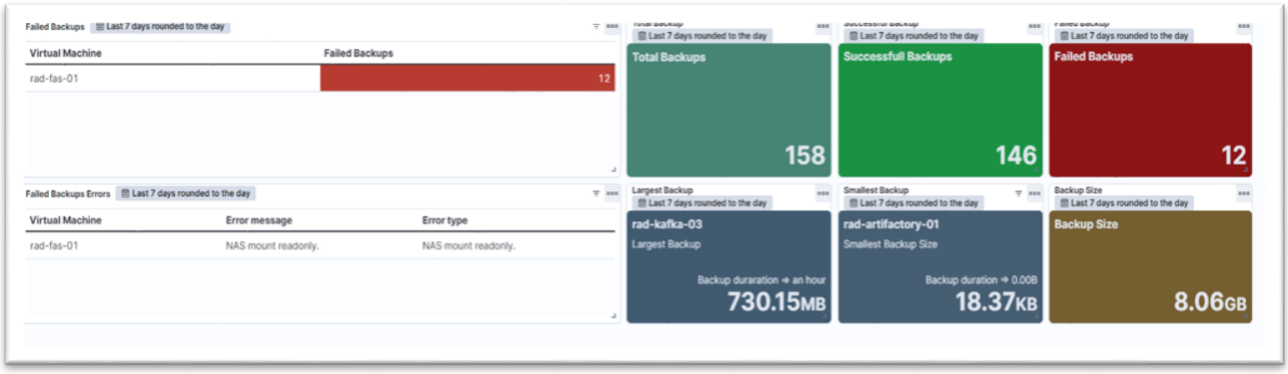
The dashboard does not just sort protocols; it also monitors whether each page is operational. It is easy to see which pages are working (Up) and which are not (Down). If a page is down, the dashboard shows you why, giving IT teams the error codes and messages they need. This helps them fix problems like server issues, DNS errors, or network problems.

By displaying both status and security information, the dashboard enables teams to respond to problems more quickly and manage web services more effectively. This ensures that everything is reliable and adheres to the rules.

Backup Monitoring Dashboard

Purpose:

To track and visualize virtual machine backup activity across the infrastructure, allowing IT teams to detect failures and performance issues early. This ensures business continuity, supports disaster recovery planning, and helps optimize storage by analyzing backup duration, size, and success rates.



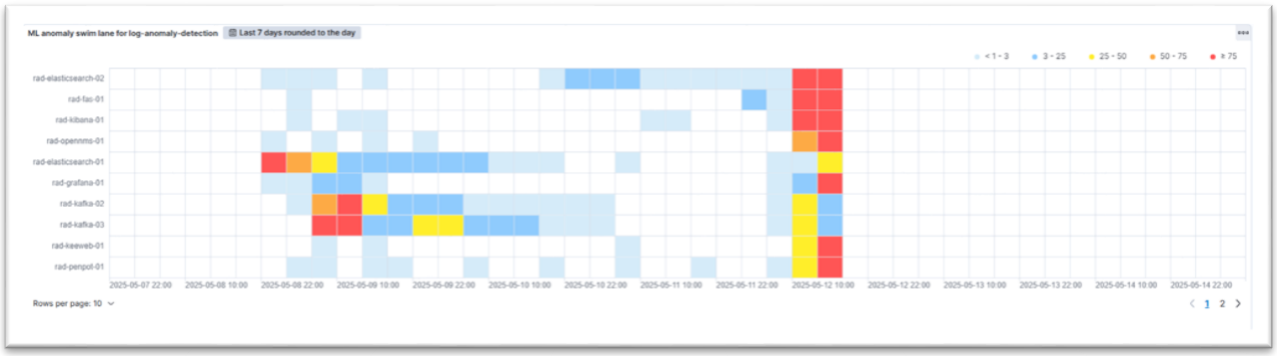
With this clear, visual layout, IT and dev teams can easily see how reliable and consistent their VM backups are. If there are repeated problems or slowdowns, they can spot them and figure out what's wrong, like storage issues, network problems, or setup mistakes.

Other metrics show the duration and size of each backup. You can see the most significant and longest virtual machine backups, along with the smaller, quicker ones. With this info, IT can tweak backup setups, make better use of storage, and keep virtual machine data safer. Knowing these patterns helps companies make their data protection plans stronger and ensure essential data is secure and easily accessible in case of an issue.

Log Anomaly Detection Dashboard

Purpose:

Identify unusual patterns and anomalies in system logs using Elasticsearch’s machine learning capabilities. This reduces reliance on static thresholds, allowing proactive detection of potential failures, security issues, or performance bottlenecks before they turn into critical incidents.



The Log Anomaly Detection Dashboard uses Elasticsearch's machine learning to spot odd patterns in

system logs automatically. Instead of just looking at the number of logs or counting errors, it studies the whole log message, putting similar messages into groups by their appearance and content.

This way, the IT team can detect subtle differences and unusual behavior that standard monitoring can miss. Unlike old systems that use set limits, Elasticsearch's models learn what's normal from old logs. These limits change as the system grows, which lowers false alerts and removes the need for manual boundary setting.

The Log Anomaly Detection Dashboard displays emerging issues as they occur. This helps monitor the system, quickly identify the cause of problems, and detect potential operational or security issues early, preventing them from worsening.

For example:

Log 1 message
Error connecting to database at 10.10.0.5:5432 – timeout

Log 2 message
Error connecting to database at 10.10.0.7:5432 – timeout

The model will group them into the same category, because they both follow
<< Error connecting to database at <IP>:5432 – timeout>>

The dashboard displays anomalies in real-time, allowing you to quickly identify when something is out of the ordinary. Developers can click on each anomaly to view the logs, see which services are impacted, and check the exact times. This lets them thoroughly investigate the problem. This detailed view lets IT find problems sooner, fix potential issues quickly, and reduce downtime.

Virtual Machines CPU and Memory Usage Dashboard

Purpose:

To provide comprehensive, real-time monitoring of virtual machine resource consumption, including CPU and memory usage. This helps IT teams prevent performance degradation, allocate resources efficiently, and detect resource-intensive workloads before they affect system stability.

This dashboard provides a detailed view of how resources are used by all virtual machines in the cloud. It shows which virtual machines are using the most CPU and memory. This lets IT find possible performance problems before they affect applications.

By monitoring resource utilization, the dashboard ensures that resources are allocated effectively and that load is balanced across the system. Historical trends and peak usage can be analyzed to plan for scaling capacity, distribute resources more efficiently, and prevent service degradation. The dashboard provides key information for maintaining the performance and reliability of cloud applications, as well as for running operations efficiently.

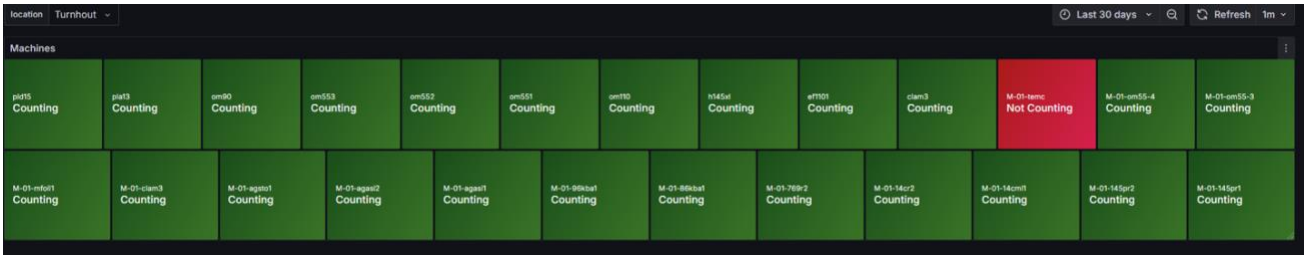
7.2 Grafana Dashboard

Smart Box Counter Monitoring Dashboard (Developer's Version)

Purpose:

To deliver detailed, granular monitoring of production machine activity for developers, enabling deep troubleshooting and analysis. This helps correlate machine downtime with specific error events, improving the accuracy and speed of root cause identification.

The Smart Box Counter dashboard provides real-time monitoring of production machines, which allows developers can monitor performance and identify problems as they occur. Each machine has its status panel with easy-to-understand color indicators. Green means the machine is running and counting, while red means it is off, has stopped working, or has a problem.



This visual status helps people quickly see how things are operating without needing to look at detailed logs. The dashboard helps with quick fixes by giving fast access to data about each machine. This permits developers to check why there is downtime or strange activity. By giving a clear, immediate view of production activity, the Smart Box Counter dashboard helps make sure manufacturing runs smoothly, lowers interruptions, and keeps output quality consistent.

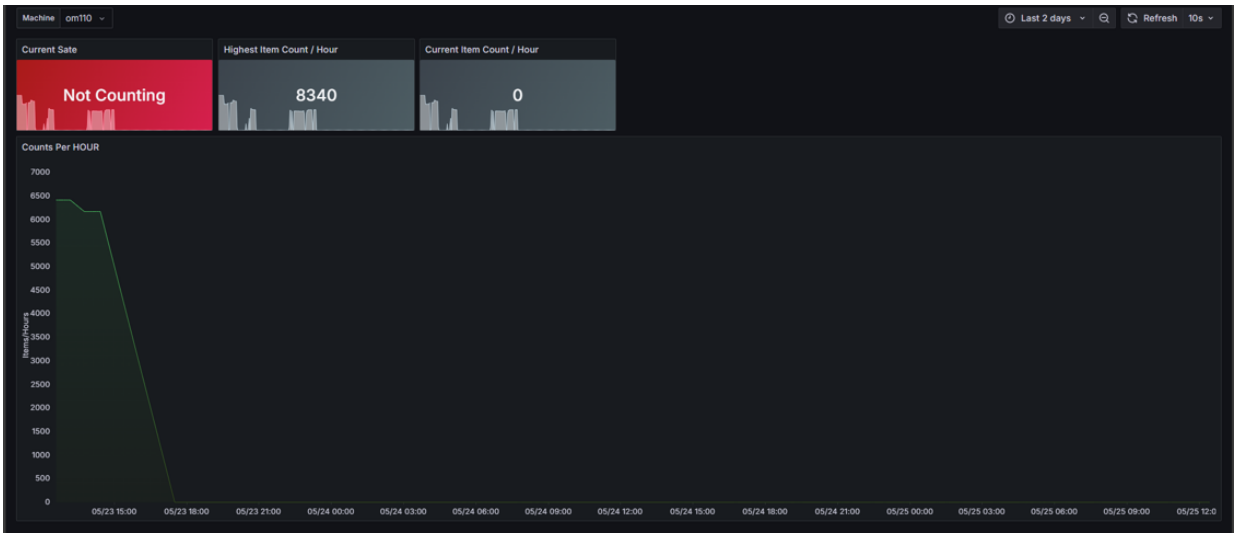
By clicking on a machine panel, developers can see detailed past trends showing item counts for specific periods. This helps find machine downtime, spot odd production issues, and check out idle times. The dashboard also helps fix problems by offering information on counting logic errors, hardware issues, and strange production patterns. This allows for quicker problem-solving and a shorter time to get things back to normal.

Smart Box Counter Monitoring Dashboard (Operators Version)

Purpose:

To provide production floor operators with a simplified, real-time overview of machine output and key performance indicators. The dashboard allows operators to quickly spot underperforming machines and take corrective action without being overwhelmed by technical details.

The operator's dashboard offers a monitoring view designed for the production floor, emphasizing ease of use. It shows hourly item counts from all machines so that operators can quickly check if rates are as expected.



Operators can take a closer look by selecting a machine to see its hourly production numbers. This helps locate machines that have issues, are down, or produce varied outputs.

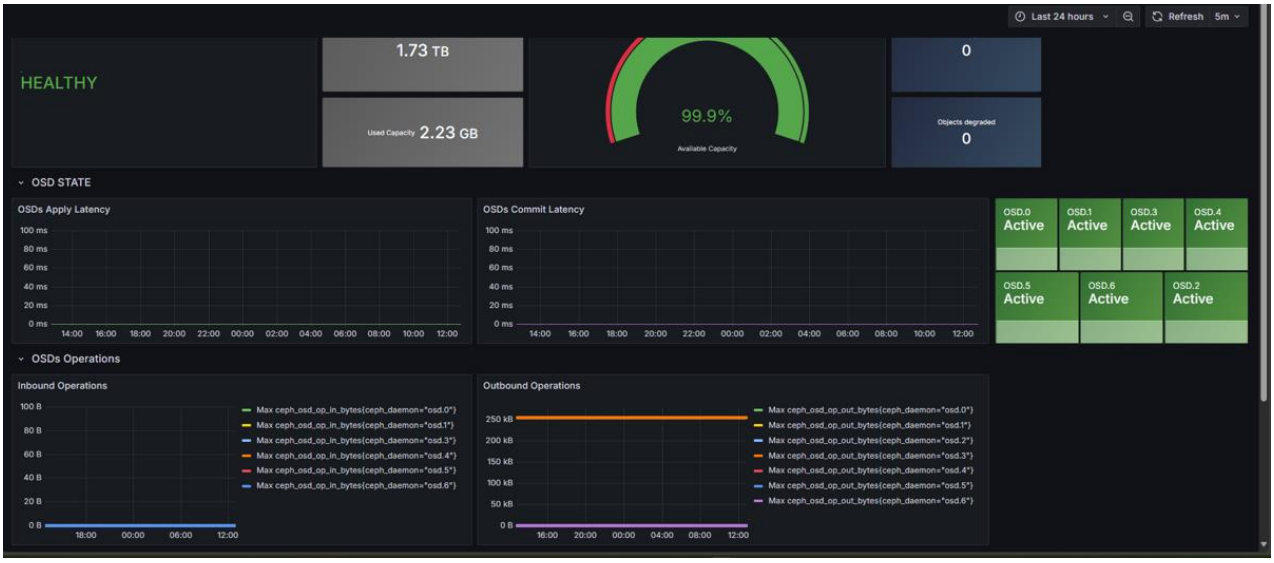
The dashboard is made to be simple and easy to use, letting staff watch output without needing special skills.

Ceph Cluster Monitoring Dashboard

Purpose:

To monitor the health, latency, and throughput of the Ceph distributed storage cluster. This enables early detection of storage issues, ensures optimal performance, and helps balance workloads across object storage devices (OSDs).

Through the Ceph dashboard, developers obtain a unified presentation of the distributed storage system performance alongside the system health status. The dashboard shows essential performance indicators, which include storage operation delays, as well as data read and write metrics, and storage cluster health status. Through this interface, developers can rapidly check system performance while detecting bottlenecks to guarantee storage reliability.



Ceph functions as an open-source software-defined storage solution that unifies object storage with block storage and file storage capabilities. The core functionality of Ceph depends on its Object Storage Daemons (OSDs) components. Each OSD controls data storage on its assigned disk while performing replication tasks and handling read/write operations. The OSDs operate together to maintain data redundancy and ensure cluster fault tolerance and high availability.

The Ceph dashboard combines real-time performance data with detailed cluster health information to provide essential monitoring, troubleshooting, and optimization capabilities for large-scale storage environments.

Kubernetes Cluster Monitoring Dashboard

Purpose:

To provide visibility into CPU and memory usage of pods across namespaces in the Kubernetes cluster. This helps IT teams optimize resource allocation, detect performance issues in specific workloads, and maintain cluster stability at scale.

The dashboard shows real-time CPU and memory usage data for all pods operating in the Kubernetes cluster, which are grouped by namespace. Developers can access detailed information about any specific pod by drilling down to view its historical resource usage trends, which helps with performance analysis

and capacity planning.

Kubernetes functions as an orchestration system that automates the deployment and scaling of containerized applications across multiple servers to achieve high availability and resilience. The project implements Kubernetes within OpenShift, which operates as a managed enterprise platform that uses Kubernetes as its base. OpenShift provides users with improved deployment workflows, enhanced security features, and integrated CI/CD capabilities on top of standard Kubernetes functionality.

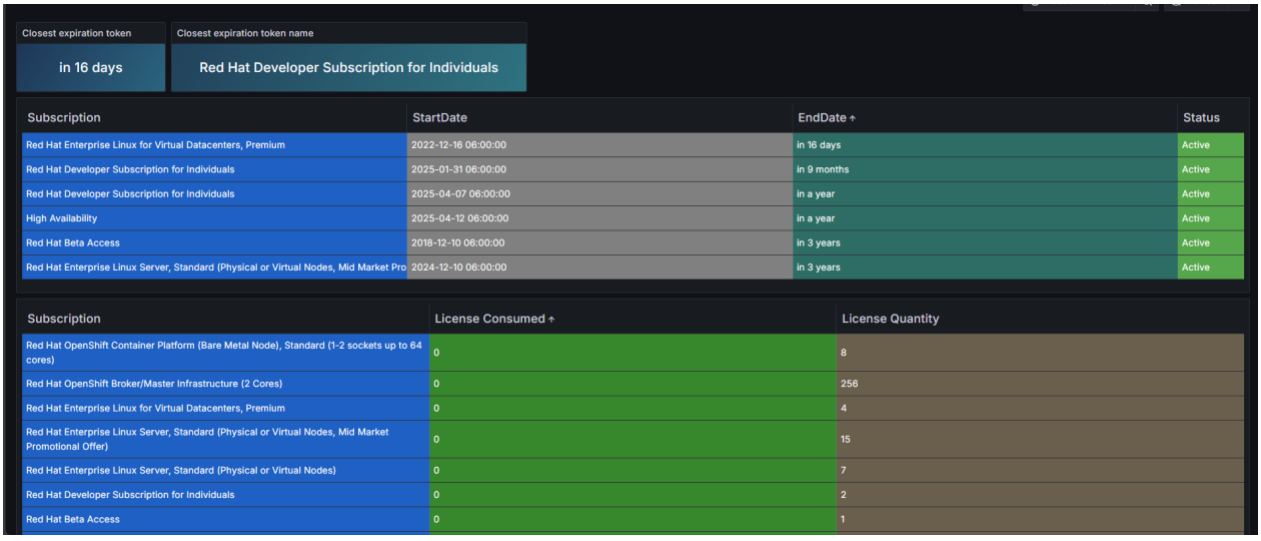
Using this information, IT departments can spot resource issues early on. They can also move workloads between nodes to balance the system's performance. Plus, they can adjust how the cluster scales based on the current needs.

Red Hat License Monitoring Dashboard

Purpose:

To track and manage Red Hat license usage and expiration dates across the organization. This ensures compliance, prevents potential service interruptions due to expired licenses, and allows IT teams to plan renewals proactively.

The dashboard shows Red Hat Enterprise Linux (RHEL) licenses for all managed systems. It displays active licenses, when they expire, and upcoming renewals. Expiring licenses are shown so the IT team can act quickly and avoid service issues.



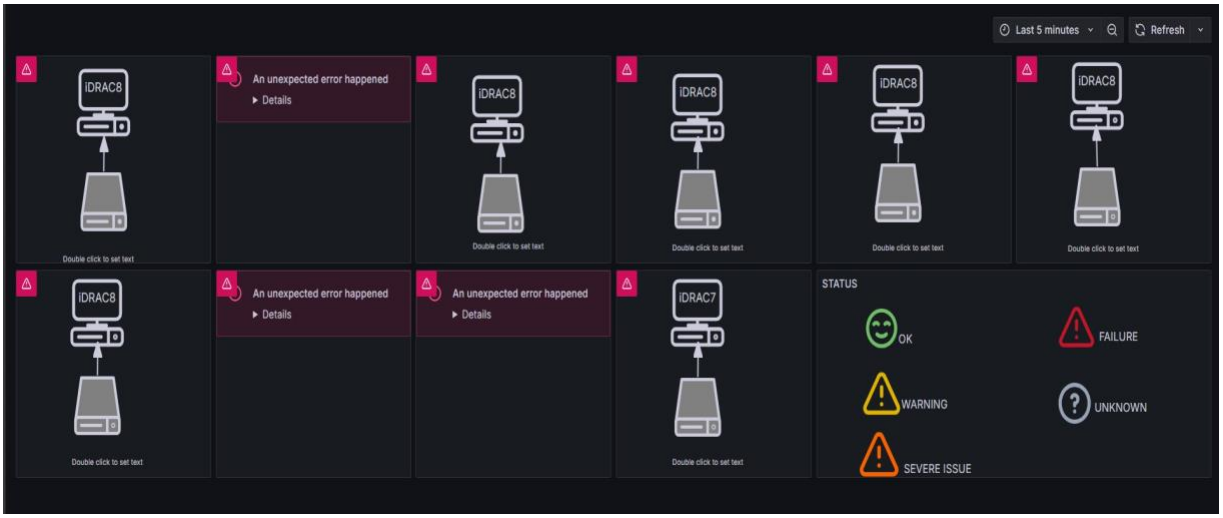
Regular use of this dashboard helps administrators renew all licenses on time. This keeps systems running smoothly and helps the company meet its licensing rules.

IDRAC Servers Monitoring Dashboard (Incomplete)

Purpose:

To monitor the health of Dell servers via IDRAC interfaces, including temperature, power, disk, and fan status. This dashboard is intended to provide IT teams with proactive insights into server hardware issues to prevent downtime and ensure reliable operation once connectivity and authentication challenges are resolved.

This dashboard collects and shows server hardware health data via IDRAC (Integrated Dell Remote Access Controller), like CPU temperature, disk use, power draw, and fan speed. IDRAC, found in Dell servers, lets administrators monitor and manage hardware remotely, even when the operating system is down. It gives essential information about system performance and stability.



During setup, connection problems kept happening when trying to link to the IDRAC data. These failures stopped the system from getting the correct hardware data, which made the dashboard not work correctly.

This showed why it's essential to check if data sources are reachable before building a dashboard. This is especially true when dealing with outside hardware systems that might need special connections, logins, or network setups.

8. Results and Benefits

Key Features

The monitoring framework enables real-time dashboard monitoring of cloud infrastructure and production systems. The system continuously gathers system metrics, logs, and activity data, which it displays to provide complete system performance insights. The system provides customized dashboard views that direct users to relevant information that remains both understandable and actionable.

Benefits for Developers

The system offers developers complete dashboards that show infrastructure status alongside resource consumption and system operational data. The dashboard system helps teams detect problems and potential issues at an early stage, which enables prompt troubleshooting and better capacity planning. The system provides developers with precise real-time system behavior data, which allows them to make better choices and enhance system reliability.

Benefits for Production Staff

Production staff can monitor machine performance and production counts through simplified dashboards, requiring no technical knowledge. The system enables staff to operate smoothly while they quickly address operational changes and issues. The framework presents information in a simple format, which helps maintain efficient production operations and reduce downtime.

Overall Impact

The monitoring framework enhances existing monitoring systems by delivering immediate alerts about system failures, resource deficiencies, and abnormal events. The system enables technical and operational teams to handle infrastructure better, which results in enhanced system management, operational efficiency, and improved decision-making. The framework enhances production environment reliability through its ability to detect issues promptly so organizations can respond effectively.

9. Conclusion

Internship Overview

During this internship, a monitoring system was developed and implemented for Van Genechten Packaging's cloud and IoT infrastructure. The project involved the full dataset process, including collection from multiple sources, methods for capturing and transforming data, and enhancing data quality. In addition, dashboards were created to provide real-time visualization of system status, allowing both technical and operational teams to monitor infrastructure effectively.

Challenges and Learning

Several technical challenges emerged during the project, such as optimizing dashboard performance and planning an efficient data pipeline. Addressing these issues required research and iterative improvements. This process provided valuable experience in troubleshooting complex technical problems and designing effective, scalable systems.

Skills Development

The internship offered an opportunity to apply theoretical knowledge from academic studies to real-world scenarios. Skills in monitoring, data handling, and visualization were significantly enhanced. Moreover, the experience strengthened problem-solving abilities, critical thinking, and adaptability skills essential for success in IT and data-focused roles.

Overall Reflection

Overall, the internship was highly beneficial in bridging academic learning with practical application. It provided hands-on experience in system monitoring and data management while reinforcing key professional competencies required in modern IT environments.

Reference list

Coralogix. (2024). *Elasticsearch vs. OpenSearch – Key differences*. Retrieved from https://coralogix.com/guides/elasticsearch/elasticsearch-vs-opensearch-key-differences/

MetricFire. (2023). *Grafana vs. Power BI*. Retrieved from https://www.metricfire.com/blog/grafana-vs-power-bi/

Services, A. W. (2023). *What is the ELK Stack?* Retrieved from https://aws.amazon.com/what-is/elk-stack

Signoz. (2023). *Prometheus vs Elasticsearch stack – Key concepts, features, and differences*. Retrieved from https://signoz.io/blog/prometheus-vs-elasticsearch/

Simplilearn. (2023). Retrieved from Kafka vs RabbitMQ: Key Differences & Features Explained: https://www.simplilearn.com/kafka-vs-rabbitmq-article